數據庫知識點與語法大全

一、SQL 語言類型

1. **DDL (Data Definition Language) - 數據定義語言**

用於定義和管理數據庫結構

```
-- 創建數據庫
CREATE DATABASE 數據庫名稱;

-- 創建表
CREATE TABLE 表名稱 (
    列1 數據類型 [約束],
    列2 數據類型 [約束],
    ...
    [表級約束]
);

-- 修改表
ALTER TABLE 表名稱
ADD 列名 數據類型;

ALTER TABLE 表名稱
DROP COLUMN 列名;

ALTER TABLE 表名稱
MODIFY COLUMN 列名 新數據類型;

-- 刪除表
DROP TABLE 表名稱;

-- 刪除數據庫
DROP DATABASE 數據庫名稱;
```

2. **DML (Data Manipulation Language) - 數據操作語言**

用於操作數據

```
-- 插入數據
INSERT INTO 表名稱 (列1, 列2, ...)
VALUES (值1, 值2, ...);

-- 更新數據
UPDATE 表名稱
SET 列1 = 值1, 列2 = 值2
WHERE 條件;
```

```
-- 刪除數據
DELETE FROM 表名稱
WHERE 條件;
```

3. **DQL (Data Query Language) - 數據查詢語言**

主要用於查詢數據

```
SELECT [DISTINCT] 列1, 列2, ...
FROM 表名稱
[JOIN 表2 ON 條件]
[WHERE 條件]
[GROUP BY 分組列]
[HAVING 分組條件]
[ORDER BY 排序列 [ASC|DESC]]
[LIMIT 數量];
```

4. **DCL (Data Control Language) - 數據控制語言**

用於控制存取權限

```
-- 授予權限
GRANT 權限列表 ON 數據庫.表 TO 用戶;

-- 撤銷權限
REVOKE 權限列表 ON 數據庫.表 FROM 用戶;
```

二、數據類型

數值類型

- `INT` / `INTEGER` - 整數
- `DECIMAL(p, s)` - 精確數值，p為總位數，s為小數位數
- `FLOAT` - 浮點數
- `DOUBLE` - 雙精度浮點數

字符串類型

- `CHAR(n)` - 固定長度字符串
- `VARCHAR(n)` - 可變長度字符串
- `TEXT` - 長文本

日期時間類型

- `DATE` - 日期 (YYYY-MM-DD)
- `TIME` - 時間 (HH:MM:SS)

- `DATETIME` - 日期時間
- `TIMESTAMP` - 時間戳

布爾類型

- `BOOLEAN` / `BOOL` - 布爾值

三、約束 (Constraints)

```
CREATE TABLE students (
    -- 主鍵約束
    學號 INT PRIMARY KEY,

    -- 唯一約束
    身份證號 VARCHAR(20) UNIQUE,

    -- 非空約束
    姓名 VARCHAR(50) NOT NULL,

    -- 檢查約束
    成績 INT CHECK (成績 >= 0 AND 成績 <= 100),

    -- 默認值
    創建時間 DATETIME DEFAULT CURRENT_TIMESTAMP,

    -- 外鍵約束
    班級代號 INT,
    FOREIGN KEY (班級代號) REFERENCES classes(班級代號),

    -- 自動遞增
    id INT AUTO_INCREMENT
);
```

四、SELECT 查詢詳解

基本查詢

```
-- 選擇所有列
SELECT * FROM 表名;

-- 選擇特定列
SELECT 列1, 列2 FROM 表名;

-- 使用別名
SELECT 列1 AS 別名1, 列2 AS 別名2 FROM 表名;
SELECT 列1 別名1 FROM 表名;

-- 去重複
SELECT DISTINCT 列名 FROM 表名;
```

WHERE 條件篩選

```sql
-- 比較運算符
SELECT * FROM products
WHERE 價格 > 100;

SELECT * FROM students
WHERE 成績 >= 60;

-- 邏輯運算符
SELECT * FROM employees
WHERE 部門 = '銷售部' AND 工齡 > 5;

SELECT * FROM products
WHERE 類別 = '電子' OR 類別 = '服裝';

SELECT * FROM students
WHERE NOT 成績 < 60;

-- IN 運算符
SELECT * FROM products
WHERE 類別 IN ('電子', '書籍', '食品');

-- BETWEEN 運算符
SELECT * FROM orders
WHERE 金額 BETWEEN 100 AND 500;

-- LIKE 模糊查詢
SELECT * FROM customers
WHERE 姓名 LIKE '張%';  -- 張開頭

SELECT * FROM customers
WHERE 電話 LIKE '%1234%';  -- 包含1234

SELECT * FROM products
WHERE 名稱 LIKE '_機';  -- 兩個字，第二字是"機"

-- NULL 值判斷
SELECT * FROM employees
WHERE 電話 IS NULL;

SELECT * FROM employees
WHERE 電話 IS NOT NULL;
```

聚合函數

```sql
-- 統計行數
SELECT COUNT(*) FROM 表名;
SELECT COUNT(DISTINCT 列名) FROM 表名;
```

```sql
-- 求和
SELECT SUM(金額) FROM orders;

-- 平均值
SELECT AVG(成績) FROM students;

-- 最大值
SELECT MAX(價格) FROM products;

-- 最小值
SELECT MIN(價格) FROM products;

-- 分組統計
SELECT 班級, AVG(成績) AS 平均成績
FROM students
GROUP BY 班級;

-- 分組後篩選
SELECT 班級, AVG(成績) AS 平均成績
FROM students
GROUP BY 班級
HAVING AVG(成績) > 60;
```

連接查詢 (JOIN)

```sql
-- 內連接 (INNER JOIN)
SELECT s.學號, s.姓名, c.班級名稱
FROM students s
INNER JOIN classes c ON s.班級代號 = c.班級代號;

-- 左連接 (LEFT JOIN)
SELECT s.學號, s.姓名, c.班級名稱
FROM students s
LEFT JOIN classes c ON s.班級代號 = c.班級代號;

-- 右連接 (RIGHT JOIN)
SELECT s.學號, s.姓名, c.班級名稱
FROM students s
RIGHT JOIN classes c ON s.班級代號 = c.班級代號;

-- 全連接 (FULL OUTER JOIN - 部分數據庫支持)
SELECT s.學號, s.姓名, c.班級名稱
FROM students s
FULL OUTER JOIN classes c ON s.班級代號 = c.班級代號;

-- 交叉連接 (CROSS JOIN)
SELECT * FROM table1 CROSS JOIN table2;

-- 自連接
SELECT e1.姓名 AS 員工, e2.姓名 AS 主管
```

```
FROM employees e1
LEFT JOIN employees e2 ON e1.主管編號 = e2.員工編號;
```

子查詢 (Subquery)

```
-- 在 WHERE 中使用
SELECT * FROM products
WHERE 價格 > (SELECT AVG(價格) FROM products);

-- 在 SELECT 中使用
SELECT 姓名,
       (SELECT COUNT(*) FROM orders WHERE 客戶編號 = c.編號) AS 訂單數
FROM customers c;

-- 在 FROM 中使用
SELECT 部門, AVG(平均成績) AS 部門平均
FROM (
    SELECT 部門, AVG(成績) AS 平均成績
    FROM employees
    GROUP BY 部門
) AS 部門統計
GROUP BY 部門;

-- IN 子查詢
SELECT * FROM products
WHERE 供應商編號 IN (
    SELECT 編號 FROM suppliers WHERE 地區 = '台灣'
);

-- EXISTS 子查詢
SELECT * FROM customers c
WHERE EXISTS (
    SELECT 1 FROM orders o
    WHERE o.客戶編號 = c.編號 AND o.金額 > 1000
);
```

五、視圖 (Views)

```
-- 創建視圖
CREATE VIEW 視圖名稱 AS
SELECT 列1, 列2, ...
FROM 表名
WHERE 條件;

-- 使用視圖
SELECT * FROM 視圖名稱;

-- 修改視圖
CREATE OR REPLACE VIEW 視圖名稱 AS
```

```
SELECT 列1, 列2, ...
FROM 表名
WHERE 條件;

-- 刪除視圖
DROP VIEW 視圖名稱;
```

六、索引 (Indexes)

```
-- 創建索引
CREATE INDEX 索引名稱 ON 表名 (列名);

-- 創建唯一索引
CREATE UNIQUE INDEX 索引名稱 ON 表名 (列名);

-- 創建複合索引
CREATE INDEX 索引名稱 ON 表名 (列1, 列2);

-- 刪除索引
DROP INDEX 索引名稱 ON 表名;
```

七、事務控制

```
-- 開始事務
BEGIN TRANSACTION;  -- 或 START TRANSACTION;

-- 提交事務
COMMIT;

-- 回滾事務
ROLLBACK;

-- 設置保存點
SAVEPOINT 保存點名稱;

-- 回滾到保存點
ROLLBACK TO 保存點名稱;
```

八、函數和運算符

字符串函數

```
SELECT CONCAT(姓, ' ', 名) AS 全名 FROM employees;
SELECT UPPER(姓名) FROM students;
SELECT LOWER(姓名) FROM students;
SELECT LENGTH(字符串) FROM table;
```

```
SELECT SUBSTRING(字符串, 起始位置, 長度) FROM table;
SELECT TRIM('  字符串  ') FROM table;
SELECT REPLACE(文本, '舊', '新') FROM table;
```

數學函數

```
SELECT ROUND(數值, 小數位數) FROM table;
SELECT CEIL(數值) FROM table;  -- 向上取整
SELECT FLOOR(數值) FROM table; -- 向下取整
SELECT ABS(數值) FROM table;   -- 絕對值
SELECT MOD(被除數, 除數) FROM table; -- 取餘數
SELECT POWER(底數, 指數) FROM table; -- 冪運算
```

日期函數

```
SELECT NOW();  -- 當前日期時間
SELECT CURDATE();  -- 當前日期
SELECT CURTIME();  -- 當前時間
SELECT DATE(日期時間) FROM table;
SELECT TIME(日期時間) FROM table;
SELECT YEAR(日期) FROM table;
SELECT MONTH(日期) FROM table;
SELECT DAY(日期) FROM table;
SELECT DATE_ADD(日期, INTERVAL 1 DAY) FROM table;
SELECT DATEDIFF(日期1, 日期2) FROM table;
```

條件函數

```
-- CASE 表達式
SELECT 姓名, 成績,
    CASE
        WHEN 成績 >= 90 THEN '優'
        WHEN 成績 >= 80 THEN '良'
        WHEN 成績 >= 60 THEN '及格'
        ELSE '不及格'
    END AS 等級
FROM students;

-- IF 函數 (MySQL)
SELECT 姓名, IF(成績 >= 60, '及格', '不及格') AS 結果
FROM students;

-- COALESCE 函數 (返回第一個非NULL值)
SELECT COALESCE(電話, '未提供') AS 聯繫電話
FROM customers;
```

```
-- NULLIF 函數 (如果兩個值相等返回NULL)
SELECT NULLIF(列1, 列2) FROM table;
```

九、正規化 (Normalization)

第一正規化 (1NF)

- 每個欄位只包含單一值
- 沒有重複的欄位

第二正規化 (2NF)

- 符合 1NF
- 所有非主鍵欄位完全依賴於主鍵

第三正規化 (3NF)

- 符合 2NF
- 所有非主鍵欄位之間沒有依賴關係

十、性能優化技巧

查詢優化

1. **使用索引**：在 WHERE、JOIN、ORDER BY 的欄位上創建索引
2. **避免 SELECT ***：只選擇需要的欄位
3. **適當使用 JOIN**：優先使用 INNER JOIN
4. **避免在 WHERE 中使用函數**：會使索引失效
5. **使用 LIMIT**：限制返回的行數
6. **避免子查詢**：能用 JOIN 代替就用 JOIN

設計優化

1. **選擇適當的數據類型**
2. **正規化與反正規化的平衡**
3. **分割大表**
4. **使用分區表**

十一、常見面試問題

理論題

1. **什麼是 ACID 特性？**
    - Atomicity (原子性)
    - Consistency (一致性)
    - Isolation (隔離性)
    - Durability (持久性)
2. **什麼是事務隔離級別？**

    - READ UNCOMMITTED
    - READ COMMITTED
    - REPEATABLE READ
    - SERIALIZABLE

3. **索引的種類和原理？**

    - B-Tree 索引
    - 哈希索引
    - 全文索引
    - 複合索引

實踐題

1. **如何找出重複的資料？**

```
SELECT 列1, 列2, COUNT(*)
FROM 表名
GROUP BY 列1, 列2
HAVING COUNT(*) > 1;
```

2. **如何刪除重複資料？**

```
DELETE FROM 表名
WHERE id NOT IN (
    SELECT MIN(id)
    FROM 表名
    GROUP BY 重複列
);
```

3. **如何計算排名？**

```
SELECT 姓名, 成績,
       ROW_NUMBER() OVER (ORDER BY 成績 DESC) AS 排名
FROM students;
```

十二、各數據庫差異

MySQL vs PostgreSQL vs SQL Server

1. **字符串連接**

    - MySQL: `CONCAT(str1, str2)`
    - SQL Server: `str1 + str2`
    - PostgreSQL: `str1 || str2`

2. **分頁**
    - MySQL: `LIMIT 10 OFFSET 20`
    - SQL Server: `OFFSET 20 ROWS FETCH NEXT 10 ROWS ONLY`
    - PostgreSQL: `LIMIT 10 OFFSET 20`
3. **日期函數**
    - 各數據庫的日期函數名稱和用法略有不同

十三、最佳實踐

編碼規範

```sql
-- 使用大寫關鍵字
SELECT column1, column2 FROM table_name;

-- 使用有意義的表名和欄位名
CREATE TABLE customers (
    customer_id INT PRIMARY KEY,
    customer_name VARCHAR(100) NOT NULL
);

-- 使用註釋
/*
 * 功能：查詢活躍用戶
 * 作者：XXX
 * 日期：2024-01-01
 */
SELECT * FROM users WHERE status = 'active';
```

安全考慮

1. **防止 SQL 注入**
    - 使用參數化查詢
    - 驗證輸入數據
    - 使用存儲過程
2. **權限管理**
    - 最小權限原則
    - 定期審計權限

十四、練習題示例

基礎題

```sql
-- 1. 查詢每個部門的員工人數和平均薪資
SELECT 部門,
       COUNT(*) AS 員工人數,
       AVG(薪資) AS 平均薪資
FROM employees
GROUP BY 部門;

-- 2. 查詢薪資高於部門平均薪資的員工
SELECT e1.*
FROM employees e1
WHERE e1.薪資 > (
    SELECT AVG(e2.薪資)
    FROM employees e2
    WHERE e2.部門 = e1.部門
);

-- 3. 查詢沒有訂單的客戶
SELECT c.*
FROM customers c
LEFT JOIN orders o ON c.客戶編號 = o.客戶編號
WHERE o.訂單編號 IS NULL;
```

進階題

```sql
-- 1. 查詢連續3天登錄的用戶
SELECT DISTINCT a.user_id
FROM login_log a
JOIN login_log b ON a.user_id = b.user_id
    AND b.login_date = DATE_ADD(a.login_date, INTERVAL 1 DAY)
JOIN login_log c ON a.user_id = c.user_id
    AND c.login_date = DATE_ADD(a.login_date, INTERVAL 2 DAY);

-- 2. 計算累計和
SELECT 日期, 金額,
       SUM(金額) OVER (ORDER BY 日期) AS 累計金額
FROM sales;

-- 3. 查詢每個類別銷售額前三的產品
SELECT 類別, 產品名稱, 銷售額, 排名
FROM (
    SELECT 類別, 產品名稱, 銷售額,
           ROW_NUMBER() OVER (PARTITION BY 類別 ORDER BY 銷售額 DESC) AS 排名
    FROM product_sales
) AS temp
WHERE 排名 <= 3;
```